SPRING SECURITY para LOGIN

Tipos de login:

1- Manualmente (no se recomienda porque puede romperse por fuerza bruta). Seria un buen ejercicio de programacion, implica crear entidad (tabla en la BD), repositorio, serivico y controlador (ruta web), y comprobar si el dato es correcto en capa servicio y esto daria acceso a la siguiente página del cCRUD de coches si es correcto.

2- Encriptado nativo de Spring Security (bCrypt): en la base de datos no te guarda "1234", sino un hash largo. Spring contiene la manera de verificarlo.
Ejemplo "1234" =>
'$2a$10$9m.WYtRnfkfHLVbPE9LOQuRVIIzCaxDOV.HdtaHRUeBSdDSnWpeHS'

3- Jwt Json Web Token: permite acceso más constante a la BD una vez la sesión esta abierta, auqnue puede haber complicaciones y hay que crear archivos ajva extra

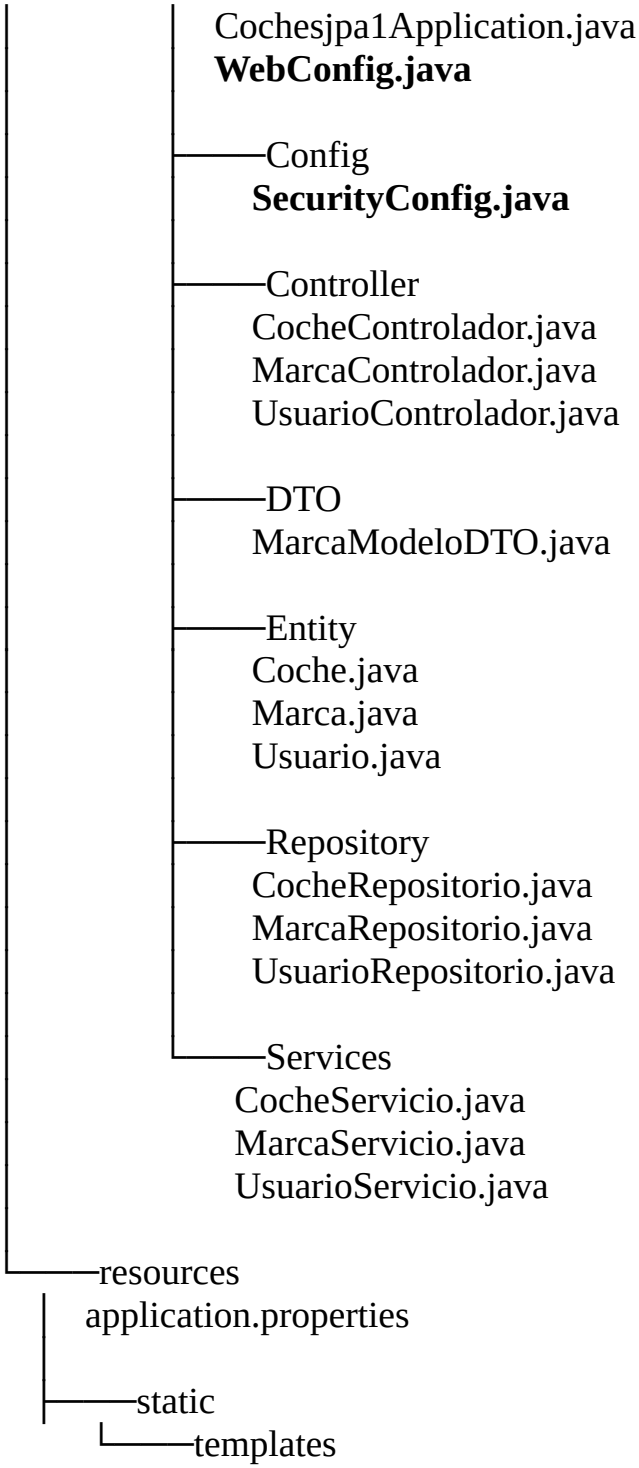
4- oAuth2: otra forma que suele incluir la anterior y permite por ejemplo entrar con tu cuenta de **Google.** El problema es darse de alta en **Google Cloud.** Hay una opción 'gratis' que en realidad te descuenta 10€ de tu cuenta. Esto es un gasto para el creador de software, no para el usuario. Aún hay que añadir más código, y también en el front.

PUNTO 2 paso a paso:
Como siempre cuando hay una tabla nueva ('usuario') y se quiere acceder desde web, crear nuevos archivos en cada capa en este orden:
1- entidad Usuario
2- JPA repositorio: añadir buscar por nombre
3- servicio: usa el repositorio y puede tener logica añadida
4- controlador: ruta de acceso al login

Además, hay que cambiar la cofiguracion de seguridad:
- los CORS ya no vale ="*", sino que hay que activar un archivo de configuracion de CORS, [cross reference servidores]
- y aparte otro archivo en carpeta de configuracion general de seguridad [encriptar]
- el pom.xml no hace falta en pricnipio tocarlo, porque ya viene la dependencia de spring security
- en application.properties tampoco hay que hacer nada

Arbol de la aplicación:

```
├─────java
│   └─────com
│       └─────ironhack
│           └─────cochesjpa1
```



(el pom.xml queda en la raíz)

- ENTITY: Usuario es entidad normal, con id, user y password

- REPO: aquí se ñade un buscador automatico por nombre de columna con:

```
Optional<Usuario> findByUsername(String username);
```

- SERVICIO: contiene la logica y implementa la seguridad (Password Encoder)

- CONTROLADOR: le indicas la ruta y método permitido, en este caso POST apra crear nuevo user y password

- añadir capa de config con archivo de configuracion (copy+paste)

- y los CORS, cambiar la linea de cors=”*” por un archivo de configuracion en la raiz ‘webconfig’:

```
.allowedOrigins(
          "http://localhost:5500",
          "http://127.0.0.1:5500"
        )
```

Esto para pruebas locales, si se sube a internet, hay que cambiar los servidores.
Y `allowCredentials(true);`
esto para enviar las credenciales con POST desde el front con fetch

En el front, pequeños cambios en el HTML (poner el contenido la pagina en un div para ocultarlo y después mostrarlo cuando entras existosamente).

E JS sí que hay que usar login/logout
`SearchParams();`

estos son los parametros de la URL.
Tambien está el ver/esconder el contenido de la pagina
En el back, en `application.properties`: , puedes añadir un temporizador de sesion: server.servlet.session.timeout=10m
(saldria de la sesion en 10 minutos)